

Facade Pattern

Từ "Facade" trong tiếng Anh nghĩa là "mặt tiền" (hoặc bề ngoài) của một tòa nhà. Trong lập trình, Facade Pattern là việc bạn xây dựng một cái "mặt tiền" đơn giản để che giấu toàn bộ sự phức tạp, rối rắm của một hệ thống phía sau. Facade Pattern là một structural design pattern, mục đích là cung cấp một interface đơn giản (simplified) để che giấu sự phức tạp của một hệ thống con (subsystem) gồm nhiều ý tưởng cốt lõi: thay vì client phải tự gọi tuần tự nhiều object con với logic phức tạp, ta tạo ra một class "Facade" đứng giữa, gói gọn toàn bộ quy trình đó lại thành một (hoặc vài) method gọi đơn giản.

Vấn đề bạn gặp phải (Khi KHÔNG CÓ Facade)

Hãy tưởng tượng bạn mua một thư viện xử lý video siêu khủng của bên thứ ba để làm app đăng video mèo. Thư viện này có hàng trăm class, hàng ngàn tính năng. Để chuyển đổi được một đoạn video ngắn, code của bạn sẽ phải:

- Khởi tạo Object A (Cấu hình âm thanh).
- Khởi tạo Object B (Cấu hình hình ảnh).
- Khởi tạo Object C (Quản lý bộ nhớ đệm).
- Gọi hàm của A, lấy kết quả truyền vào B, rồi lại truyền vào C theo đúng thứ tự.

Hậu quả: Code chính của bạn (Business Logic) bị dính chặt vào cái đồng bù nhùi của thư viện kia. Sau này thư viện đó cập nhật, code của bạn sẽ "toang" theo.

Ví dụ kinh điển trong Spring Boot: giả sử khi tạo đơn hàng (Order), bạn cần phối hợp

- `InventoryService` (kiểm tra tồn kho)
- `PaymentService` (xử lý thanh toán)
- `ShippingService` (tạo vận đơn)
- `NotificationService` (gửi email)

Nếu để Controller tự gọi từng service theo đúng thứ tự, xử lý lỗi, rollback... thì Controller sẽ phình to và lộ chi tiết nghiệp vụ ra ngoài. Giải pháp là tạo OrderFacade :

```
@Service
public class OrderFacade {
    private final InventoryService inventoryService;
    private final PaymentService paymentService;
    private final ShippingService shippingService;
    private final NotificationService notificationService;

    public OrderResult placeOrder(OrderRequest request) {
        inventoryService.reserveItems(request.getItems());
        PaymentResult payment = paymentService.charge(request.getPaymentInfo());
        shippingService.createShipment(request);
        notificationService.sendOrderConfirmation(request.getCustomerEmail());
        return new OrderResult(payment, ...);
    }
}
```

Controller lúc này chỉ cần gọi `orderFacade.placeOrder(request)` mà không cần biết bên trong có bao nhiêu service đang phối hợp. Vài điểm cần phân biệt vì hay bị nhầm:

So với Service layer trong Spring: thật ra trong thực tế, `@Service` của bạn thường đã đóng luôn vai trò Facade — nó là điểm tổng hợp logic từ nhiều Repository/Component khác để Controller gọi vào. Đây là lý do Facade là một trong những pattern bạn dùng "vô thức" hằng ngày mà không gọi tên nó. So với Adapter Pattern: Adapter dùng để chuyển đổi interface không tương thích thành tương thích (thường là 1-1, ví dụ wrap một thư viện third-party). Facade dùng để đơn giản hóa nhiều interface phức tạp thành 1 interface đơn giản hơn (thường là nhiều-thành-một). Adapter trả lời "làm sao để 2 cái khớp nhau", Facade trả lời "làm sao để dễ dùng hơn". So với Mediator Pattern: Mediator tập trung vào việc các object con giao

subsystem bên trong thường không biết tới Facade và cũng không cần giao tiếp qua nó. Lợi ích chính: giảm coupling giữa client và subsystem, dễ test (mock Facade thay vì mock 4 5 service), và làm Controller/API layer sạch hơn vì tách biệt được "API contract" với

Giải pháp từ Facade (Khi ÁP DỤNG)

Thay vì bắt app của bạn đi tương tác với từng Object A, B, C phức tạp kia, bạn tự viết ra một Class "đại diện" (gọi là Facade Class). Class này chỉ cung cấp đúng một hàm duy nhất cực kỳ đơn giản: `encode(filename, format)`.

- **Bên ngoài App của bạn):** Chỉ cần gọi đúng hàm `encode()` là xong, không cần quan tâm thế giới ngoài kia ra sao.
- **Bên trong Facade Class):** Nó sẽ âm thầm đi gọi Object A, B, C và sắp xếp chúng chạy theo đúng thứ tự.

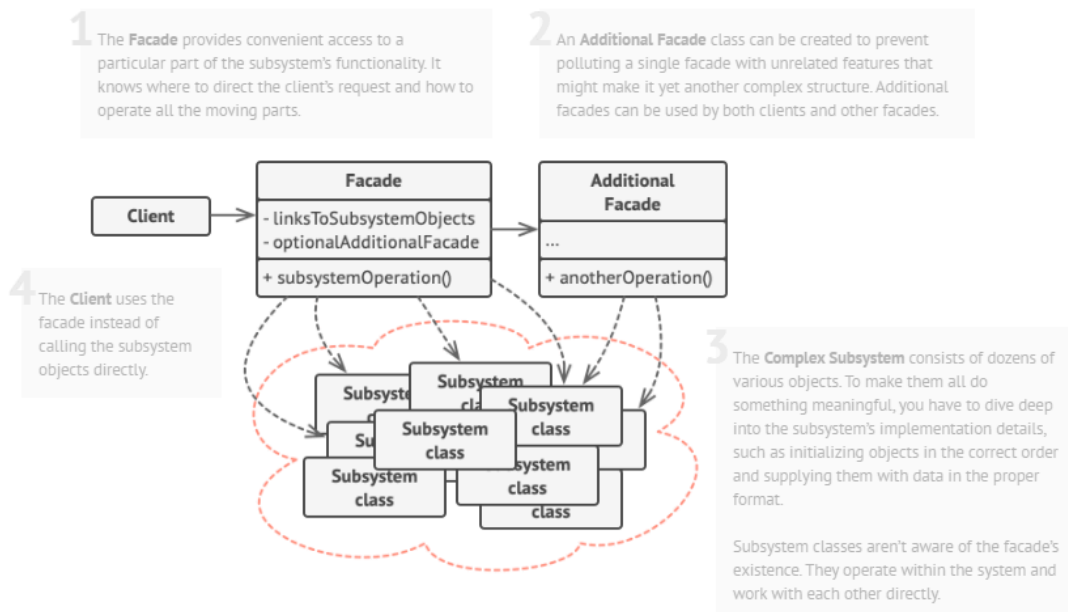
Ví dụ thực tế: Gọi đồ ăn qua Tổng đài

Nội dung tiếng Anh có ví dụ về đặt hàng qua điện thoại rất chuẩn:

- **Khi không có Facade:** Bạn muốn mua cái áo, bạn phải tự gọi cho ông thủ kho xem còn hàng không, tự gọi cho bên ngân hàng để chuyển tiền, rồi tự gọi cho ông shipper bảo qua lấy
- **Khi có Facade (Cô tổng đài viên):** Bạn chỉ cần gọi cho đúng một cô tổng đài viên và bảo: "Bán tôi cái áo size M" Cô tổng Cổ sẽ tự liên hệ kho, ngân hàng, shipper cho bạn. Bạn chỉ việc ngồi đợi nhận hàng.

Tóm lại một câu: Facade sinh ra là để làm kiếp "gánh team" – nó nhận hết sự phức tạp về mình để cho code ở các phần khác được thanh thoi, gọn gàng.

Structure



1. Client (Thành phần số 4)

- **Vai trò:** Là đoạn code gọi dịch vụ (ví dụ: **Controller** hoặc **Service** chính trong ứng dụng của bạn).
- **Cách hoạt động:** Thay vì phải tự tay khởi tạo và gọi cả một đống class phức tạp phía dưới, Client chỉ giao tiếp với duy nhất lớp **Facade** thông qua một interface đơn giản. Client hoàn toàn "mù tịt" về đống phức tạp ẩn phía sau đám mây

2. Facade - Mặt tiền chính (Thành phần số 1)

- **Vai trò:** Lớp bọc đại diện, vị cứu tinh gánh toàn bộ sự phức tạp cho Client.
- **Đặc điểm trong sơ đồ:**
 - Các thuộc tính như `linksToSubsystemObjects` : Đây là nơi Facade giữ các tham chiếu (biến) đến các class con bên dưới
 - Thuộc tính `optionalAdditionalFacade` : Tham chiếu đến một Facade bổ sung nếu cần (sẽ giải thích ở mục 3).
 - Hàm `+ subsystemOperation()` : Hàm "mặt tiền" cực kỳ ngắn gọn mà Client sẽ gọi. Bên trong ruột của hàm này mới là nơi cấu hình, sắp xếp thứ tự chạy của các class con.

3. Additional Facade - Mặt tiền bổ sung (Thành phần số 2)

- **Vai trò:** Tránh biến Facade chính thành một "God Object" (một class ôm đồm quá nhiều thứ dẫn đến phình to)
 - **Cách hoạt động:** Khi hệ thống con có thêm quá nhiều tính năng mới không liên quan trực tiếp đến luồng chính, thay vì nhét hết vào Facade cũ làm "ô nhiễm" (polluting) code, người ta tách ra thành các Additional Facade nhỏ hơn. Cả Client lẫn Facade chính đều có thể gọi đến ông Facade bổ
- 4. Complex Subsystem - Hệ thống con phức tạp (Thành phần SỐ 3)**

- **Vai trò:** Đồng bụi nhùi nằm trong đám mây màu đỏ. Đây có thể là một thư viện bên thứ ba, các service xử lý tầng thấp (implementation details), hoặc đồng logic cũ phức tạp của hệ
- **Một điểm cực kỳ quan trọng:** Sơ đồ ghi rõ "Subsystem classes aren't aware of the facade's existence" – nghĩa là các class con này hoàn toàn không biết có sự tồn tại của Facade. Tụi nó độc lập, tự chơi với nhau, nhận data và xử lý theo đúng chức năng của tụi nó. Facade chỉ là một lớp đứng từ trên cao điều phối chứ không can thiệp hay thay đổi logic